ExoHabitAI: Final Deployment & Documentation Report

For complete documentation refer - [Link]

The application has been successfully deployed using a split-stack architecture for optimal performance and minimal cold-start delays.

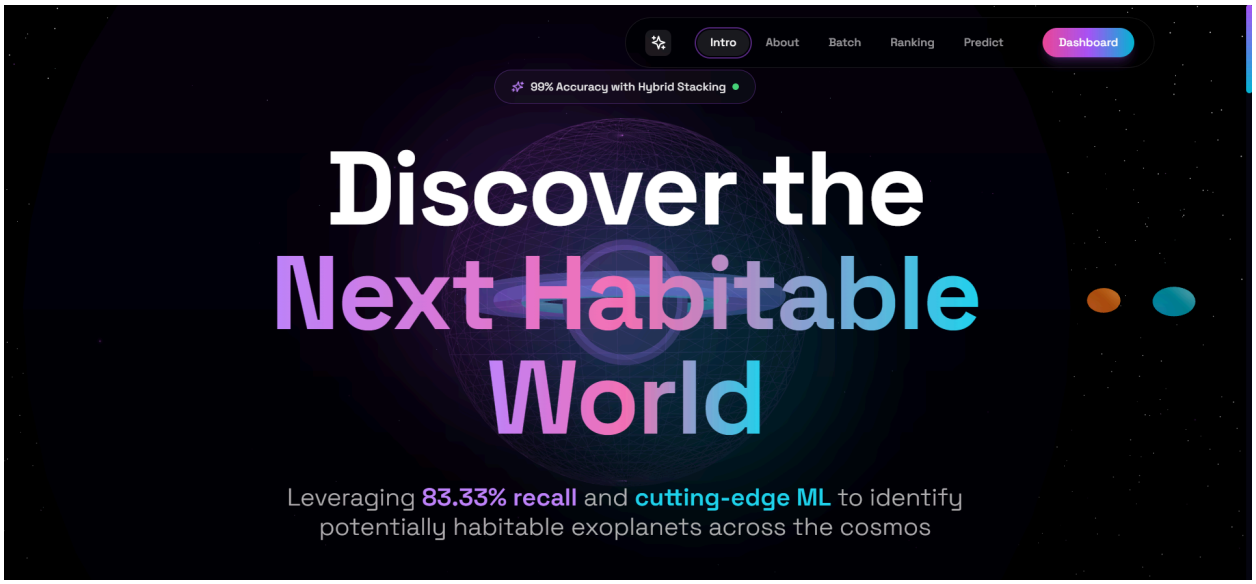
- **Production Frontend (Custom Domain):** https://exoplanetai.jainhardik06.in
- **Production Backend (Render API):** https://exohabitai-backend-79cj.onrender.com

2. GitHub Repository Link

All source code, including the React frontend, Flask backend, and the serialized `joblib` machine learning models, are committed and pushed to the repository branch:

- **Repository:** https://github.com/GKSJ19/ExoHabitAI/tree/hardikjain

3. Screenshots of Working Web App



4. API Functionality Confirmation

Extensive testing has been conducted to confirm that the deployed Flask API functions correctly and communicates seamlessly with the Vercel frontend.

- **Health Check (`GET /status`):** Returns a successful JSON response confirming the XGBoost/Random Forest model is loaded and operational.

- **Prediction Engine (`POST /predict`):** Successfully accepts 39-parameter JSON payloads, validates the input structure, processes the data through the Stacking Ensemble, and returns accurate habitability classifications.
- **Ranking Retrieval (`GET /rank`):** Successfully queries the dataset and returns the top candidates.
- **CORS Configuration:** Cross-Origin Resource Sharing (CORS) has been securely configured in `app.py` to allow the Vercel frontend to fetch data without browser security blocks.

---

5. Deployment Documentation

5.1 Deployment Architecture

The production environment splits the backend API and frontend client to optimize for speed and efficient routing.

- **Brain (Backend):** Flask & Machine Learning Model hosted on **Render** (Web Service).
- **Face (Frontend):** React & Vite UI hosted on **Vercel** (Global CDN).
- **Routing (DNS):** Custom domain routing managed via **Hostinger**.

5.2 Backend Deployment (Render)

The backend serves the XGBoost/Random Forest stacking model and exposes the REST API endpoints.

Prerequisites Configuration:

- `app.py`: Contains the main Flask application and routes.
- `requirements.txt`: Includes `flask`, `flask-cors`, `gunicorn`, `joblib`, `scikit-learn`, `xgboost`, `pandas`, `numpy`.
- `Procfile`: Contains `web: gunicorn backend.app:app` to define the WSGI server.

Render Setup Steps:

1. Navigated to Render Dashboard → New + → Web Service.
2. Connected the GitHub repository.
3. Applied the following configuration:
    - **Language/Runtime:** Python 3
    - **Root Directory:** `backend` *(Critical: Instructs Render to execute within the backend folder)*
    - **Build Command:** `pip install -r requirements.txt`
    - **Start Command:** `gunicorn app:app`
4. Deployed on the Free Tier to generate the live API URL.

5.3 Frontend Deployment (Vercel)

The frontend provides the interactive 3D UI and securely connects to the Render API.

Prerequisites Configuration:

- The API base URL was made dynamic in `frontend/src/services/api.js`:
- JavaScript
  ```
  const API_URL = import.meta.env.VITE_API_URL || "http://localhost:5000";
  ```

Vercel Setup Steps:

1. Imported the GitHub repository into Vercel as a new Project.
2. Configured the project settings:
    - **Framework Preset:** Vite
    - **Root Directory:** `frontend`
3. Injected Environment Variables:
    - **Key:** `VITE_API_URL`
    - **Value:** `https://exohabitai-backend-79cj.onrender.com` *(No trailing slash)*
4. Executed deployment to generate the initial Vercel production URL.

5.4 Custom Domain Integration (Hostinger)

Mapped the Vercel frontend to a professional custom domain.

1. **Vercel Configuration:** Added the target subdomain `exoplanetai.jainhardik06.in` in the project Domain settings to generate a CNAME verification target.
2. **Hostinger DNS Configuration:** Accessed the DNS Zone Editor for `jainhardik06.in` and added a CNAME record pointing `exoplanetai` to `71deece9f10bb709.vercel-dns-017.com.`.
3. **Verification:** Monitored Vercel until DNS propagation completed and SSL certificates were automatically provisioned.

5.5 Common Troubleshooting Addressed

During deployment, the following integration challenges were resolved:

- **Backend Path Errors (`ModuleNotFoundError`):** Resolved by explicitly setting the Render "Root Directory" to `backend` and updating the Gunicorn start command to `gunicorn app:app`.
- **Vercel Build Failures:** Fixed Vercel misidentifying the repository as a Python project by hard-setting the Framework Preset to Vite.
- **CORS Rejection:** Resolved API blocking by implementing `flask-cors` and configuring `CORS(app, resources={r"/*": {"origins": "*"}})` to allow frontend handshakes.